Soporte y tickets con IA

Reto técnico · Atención inicial, seguimiento y resolución

Construir un sistema de soporte cuyo punto de entrada sea un chat asistido por un modelo de IA. El asistente consulta un catálogo de problemas conocidos y propone soluciones. Cuando el problema no está documentado, persiste o requiere intervención del equipo, debe crear un ticket y entregar al usuario su número y enlace.

Resultado esperado: un flujo completo de atención y seguimiento, desde la consulta hasta el cierre. Los usuarios consultan sus tickets; los administradores los gestionan, comentan, adjuntan archivos y cambian su estado. Cada cambio de estado genera una notificación al usuario que creó el ticket.

01 · Chat como entrada al soporte

Al ingresar a Soporte, mostrar el chat y acceso visible a “Mis tickets”. Permitir describir el problema en lenguaje natural y agregar evidencias. Conservar la conversación para retomar el caso sin repetir la información.

El asistente identifica el área afectada, qué intentaba hacer el usuario, el resultado esperado, el error observado y el impacto. Solicitar únicamente datos faltantes útiles; no exigir información técnica que el usuario no tenga ni pedir contraseñas o tokens.

Consultar el catálogo antes de responder. Mostrar instrucciones claras, enlace al artículo utilizado y una pregunta concreta para confirmar si el problema se resolvió. No afirmar que se inspeccionó producción ni que se ejecutó una reparación si no existe una herramienta y evidencia que lo respalde.

02 · Cuándo crear un ticket

Situación	Comportamiento requerido
Existe una solución aplicable	Explicarla y pedir confirmación de resultado. Si resuelve la consulta, conservar la conversación sin crear un ticket innecesario.
La solución falla o el error persiste	Crear un ticket con los pasos ya intentados y su resultado. No repetir indefinidamente la misma sugerencia.
No hay una respuesta en el catálogo	Recoger el mínimo necesario y crear el ticket. Una respuesta improvisada del modelo no sustituye la escalación.
Se reporta una falla o se requiere intervención	Registrar el reporte como ticket y distinguir síntoma reportado de causa confirmada. La IA no debe exigir una certeza técnica imposible para escalar.
El usuario solicita ayuda humana	Permitir crear el ticket directamente y conservar el contexto disponible.

El backend confirma la creación antes de que el chat anuncie éxito. Responder con número legible, título, estado inicial **Abierto** y enlace que permita abrir el ticket con una sesión autorizada. Ante un error, conservar el borrador, informar el fallo y permitir reintento sin duplicados.

Catálogo y tickets

Conocimiento mantenible y ciclo de atención claro

03 · Catálogo de problemas conocidos

Crear una sección administrable de artículos con título, categoría, síntomas, plataforma o versión aplicable, pasos de diagnóstico, solución, condiciones de escalación, etiquetas, estado de publicación y fecha de revisión. Permitir búsqueda, edición, publicación y archivado.

Incluir un conjunto inicial de ejemplos de prueba: problemas de acceso, permisos, funcionamiento de pantallas, carga de archivos y comportamiento inesperado. Identificarlos como ejemplos; no presentar procedimientos no verificados como soluciones oficiales de PBG.

El chat utiliza artículos publicados y vigentes para el caso. Guardar qué artículo y versión se usaron. Si hay un incidente conocido todavía sin solución, informar su estado y crear o vincular el ticket del usuario al incidente, sin exponer casos privados de otros usuarios.

Permitir que un administrador prepare un artículo a partir de un ticket resuelto, revise su contenido y lo publique. No incorporar automáticamente comentarios privados, adjuntos o datos personales al catálogo.

04 · Información y sección de tickets

Cada ticket debe tener ID interno y número visible, título, descripción, solicitante, categoría, impacto/prioridad, estado, responsable opcional, fechas de creación y actualización, conversación de origen, pasos intentados, adjuntos y registro de eventos. Mantener el reporte original separado del resumen generado por IA.

La sección de tickets debe ofrecer vistas **Abiertos, En revisión y Cerrados**, búsqueda, filtros, ordenación y paginación. Mostrar actualizaciones recientes y elementos sin leer. En el detalle, presentar descripción, comentarios, evidencias, historial y estado actual.

05 · Estados y reglas de transición

Estado	Significado y acciones
Abierto	Estado inicial. Pendiente de que el equipo tome el caso.
En revisión	Un administrador está investigando o atendiendo el caso. Puede asignar responsable, solicitar información y comentar.
Cerrado	El administrador registra un resumen de resolución o motivo de cierre. No basta con una sugerencia de la IA para cerrar.

Permitir Abierto → En revisión → Cerrado. Admitir Abierto → Cerrado con motivo explícito, por ejemplo duplicado o reporte inválido. Si un ticket cerrado vuelve a fallar, permitir al solicitante reabrirlo como Abierto, conservando el historial y notificando el cambio.

Las asignaciones, prioridades y etiquetas complementan los tres estados, sin reemplazarlos. Evitar que dos administradores sobrescriban cambios simultáneos: verificar la versión del ticket y mostrar conflictos recuperables. Un clic repetido con el mismo estado no es una nueva transición.

Gestión y notificaciones

Colaboración entre usuario y equipo de soporte

06 · Experiencia del solicitante

El usuario puede consultar únicamente sus tickets, responder comentarios públicos, adjuntar evidencias, ver historial y resolución, marcar notificaciones como leídas y reabrir un caso. Debe poder acceder al ticket desde el enlace del chat y desde “Mis tickets”.

Un agente o dueño de agencia no obtiene acceso a tickets de otros por compartir agencia. Para este reto, solo el rol administrador gestiona todos los tickets dentro de su organización autorizada. Los clientes de agentes no son usuarios directos del sistema de soporte.

07 · Gestión administrativa

Permitir ver tickets de todos los solicitantes autorizados, filtrar por estado, prioridad, categoría, responsable y antigüedad; asignar responsables; cambiar estado; comentar y adjuntar archivos. Mostrar casos sin atender y tiempo desde la última respuesta.

Distinguir claramente **comentarios públicos**, visibles para el solicitante, de **notas internas**, visibles solo para administradores. La visibilidad también aplica a sus adjuntos y a las respuestas de API. Las notas internas no deben aparecer en notificaciones al usuario ni en respuestas del asistente.

Incluir prioridades configurables y una vista de seguimiento con tiempos de primera respuesta y resolución. Definir las fórmulas y cómo se tratan reaperturas. Si se implementan objetivos de atención o vencimientos, hacerlos configurables y no prometer plazos no establecidos.

08 · Notificar cada cambio de estado

Por cada transición guardada, crear una notificación persistente para el solicitante con número de ticket, estado anterior, nuevo estado, fecha y enlace. Informar también creación, comentarios públicos del equipo y solicitudes de información. Avisar al equipo cuando el usuario responda o reabra un caso.

Implementar bandeja de notificaciones con contador y leído/no leído. La notificación debe permanecer aunque el usuario no esté conectado; puede mostrarse en tiempo real o mediante actualización periódica. Abrir su enlace vuelve a verificar permisos.

Guardar transición y evento de notificación de forma transaccional. Procesar entregas con reintentos e idempotencia, evitando pérdidas o duplicados por fallos. Registrar entregada, pendiente o fallida por canal. El correo es un canal adicional opcional; en el reto debe probarse con un buzón local, sin enviar correos reales.

09 · Archivos e historial

Permitir imágenes y documentos con lista de tipos admitidos y límites documentados. Validar contenido y tamaño en servidor, controlar archivos peligrosos y mostrar nombre, autor, fecha y visibilidad. Guardar metadatos en PostgreSQL y binarios en almacenamiento de objetos.

Los archivos no deben ser públicos: autorizar cada descarga o usar enlaces firmados de corta duración después de verificar permisos. Conservar auditoría de estados, asignaciones y comentarios. Si se retira una evidencia o corrige un comentario, conservar trazabilidad según la política documentada.

Arquitectura y API

Asistencia confiable, datos persistentes y acceso controlado

10 · Base técnica

Extender la base del repositorio: React y TypeScript, Fastify y TypeScript, PostgreSQL 16 y Docker Compose. Reutilizar login y sesión. Usar S3 o almacenamiento compatible para adjuntos; en desarrollo puede incluirse MinIO. Los correos opcionales se prueban en Mailpit u otro buzón local.

Agregar migraciones para conversaciones, mensajes, artículos y versiones, tickets, comentarios, adjuntos, eventos, notificaciones y tareas pendientes. Revisar primero el esquema existente de soporte para decidir qué reutilizar. Documentar la decisión y conservar la estructura y datos entregados.

Usar un worker con trabajos persistentes para tareas lentas o reintentos. PostgreSQL es la fuente de verdad. El proyecto debe arrancar con Compose, ejecutar migraciones y cargar los ejemplos necesarios de forma reproducible, sin modificar servicios de producción.

11 · Modelo de IA y herramientas

El participante elige un proveedor y modelo, documenta costo y limitaciones, y configura las claves en el backend mediante variables de entorno. Implementar recuperación del catálogo y herramientas limitadas para consultar artículos, crear tickets y consultar casos autorizados.

Validar en servidor argumentos, identidad, permisos y resultado de cada herramienta. El modelo no define el solicitante ni los privilegios. No debe cambiar estados o cerrar tickets por su cuenta ni generar IDs o enlaces como si fueran confirmaciones del backend.

Tratar mensajes, artículos y adjuntos como contenido, no como instrucciones con autoridad para ejecutar acciones. Reducir los datos enviados al modelo; excluir secretos, notas internas y casos ajenos. Documentar proveedor y tratamiento de datos; usar ejemplos ficticios hasta contar con autorización para información real.

Si falla el proveedor, permitir crear el ticket mediante un formulario y conservar la conversación. Mostrar límites y tiempos de espera. No bloquear el acceso al soporte porque la IA no responda.

12 · Endpoints propuestos

Chat: POST /support/conversations; POST /support/conversations/:id/messages; GET /support/conversations/:id. **Catálogo:** GET /support/articles; endpoints administrativos para crear, versionar y publicar artículos.

Tickets: POST/GET /tickets; GET /tickets/:id; POST /tickets/:id/comments; POST /tickets/:id/attachments; PATCH /tickets/:id/status; PATCH /tickets/:id/assignee; POST /tickets/:id/reopen.

Notificaciones: GET /notifications; PATCH /notifications/:id/read. Definir descarga autorizada de adjuntos y contratos de paginación, filtros y errores. Documentar la API mediante OpenAPI o equivalente.

Aplicar permisos a listados, detalles, historial, conversaciones y archivos. Usar consultas parametrizadas, límites de uso y claves de idempotencia en creación y acciones con efectos. Cada reintento de una solicitud debe recuperar el resultado previo cuando ya se completó.

Repositorio y entrega

Arranque, usuarios de prueba y criterios de aceptación

13 · Repositorio e instalación

Repositorio privado: github.com/pbg-copilot/pbg-reto
Documentación: README. Solicitar acceso. Trabajar en una rama independiente y entregar mediante pull request; no hacer merge ni desplegar en producción.

```
git clone https://github.com/pbg-copilot/pbg-reto.git
cd pbg-reto
git switch -c feature/soporte-tickets-ia
docker compose up -d --build
```

Frontend: http://localhost:8088 · **API:** http://localhost:8008
Health: http://localhost:8008/health. La carga inicial de PostgreSQL es automática y tarda varios minutos. No requiere acceso a AWS.

La base ya incluye POST /auth/get-toke, GET /auth/me y POST /auth/logout. get-toke recibe usuario y contrasena. Mantener su contrato. La copia conserva datos personales: para los artículos y tickets de demostración, utilizar contenido ficticio.

Agente: agente099@reto.example
Administrador: agente009@reto.example
Dueño de agencia: agente016@reto.example

Contraseña local de todos: 123456. El dueño de agencia es otro solicitante; no tiene acceso a tickets ajenos ni administración de soporte.

14 · Casos mínimos de demostración

• Consulta conocida resuelta con artículo y confirmación del usuario, sin ticket innecesario.

• Problema desconocido o persistente: ticket Abierto con contexto y enlace. Un reintento no lo duplica.

• El administrador revisa, comenta, adjunta y cierra. Cada transición notifica al solicitante, también si estaba desconectado.

• El usuario responde y reabre; el historial se conserva. Notas internas y sus adjuntos permanecen ocultos para el solicitante.

• Se rechaza el acceso a tickets y adjuntos ajenos. Instrucciones maliciosas no amplían permisos.

• Caída del modelo, fallo de entrega y cambios concurrentes se recuperan. El soporte sigue disponible sin IA.

15 · Entregables y evaluación

Entregar código, migraciones, catálogo demo, pruebas, documentación de API, matriz de permisos, configuración sin secretos y README con arranque completo. Incluir demostración, costos del modelo, límites y decisiones técnicas. Evaluar utilidad de la asistencia, escalación correcta, aislamiento, notificaciones confiables, recuperación ante errores y reproducibilidad.